Formation Kubernetes

Session 4 — Storage et Persistance

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session

Le problème

Pods éphémères = données perdues

10 min

PV / PVC / StorageClass

La trio de la persistance

15 min

StatefulSet vs Deployment

Identités stables, volume par pod

10 min

PostgreSQL en K8s

Architecture avec PVC et init containers

10 min

Démo live RFLKT

Cloud SQL + proxy pattern

10 min

Travaux pratiques

PostgreSQL, StatefulSet + PVC

1h

Recap — Session 3

Ingress Controller

Point d'entrée unique pour le trafic externe

Traefik

Reverse proxy moderne, routing intelligent, middlewares

IngressRoute

CRD Traefik pour matcher Host, Path, Headers

Middlewares

Pipeline : rate-limit, headers, auth, stripPrefix

Aujourd'hui : le problème orthogonal — comment persister les données ?

Le problème : éphémérité vs persistance

Pod 1 vivant

Container

Filesystem éphémère

- Pod2 redémarré
- Nouveau filesystem
- DATA PERDUE ❌

Data : /data en RAM

⚠ Pod tombe (OOM, crash)

Pod 1 + PVC

Container

/data → PVC

- Pod2 redémarré
- PVC remontée
- DATA SAUVÉE ✓

PV (Persistent Volume)

❌ Pod tombe + redémarre

La solution : découpler le cycle de vie du pod de la durée de vie des données

PV / PVC / StorageClass — La trio magique

PersistentVolume (PV)

- Ressource K8s du cluster
- Abstraction du stockage réel
- Admin crée (déclarativ)
- Propriété : ReclaimPolicy
- Ex: GCP PD, NFS, iSCSI

PersistentVolumeClaim (PVC)

- Requête de stockage
- App demande N Gi
- K8s cherche un PV qui fit
- Binding automatique
- Cycle de vie = Pod/StatefulSet

StorageClass

- Modèle de provisioning
- Provisionnement dynamique
- Paramètres : taille, perf, type
- Ex: standard-rwo (GKE)
- Replaces PV manuel

Pod → PVC (demande) → StorageClass → PV (création) → Disque réel

Détails clés : Binding, Reclaim Policy

PVC ↔ PV Binding

1. PVC créée : capacity: 10Gi, storageClass: standard-rwo
2. K8s cherche PV ou StorageClass
3. StorageClass → PV créé (10Gi GCP disk)
4. PVC ↔ PV bindé (1-to-1)
5. Pod peut mount (volumeMounts/volumes)

⚠ Une fois bindée, la PVC reste liée au PV

Même si le pod meurt.

ReclaimPolicy (quand PVC supprimée)

Delete (défaut)

PV supprimé, disque supprimé → DATA PERDUE

Retain

PV conservé, disque conservé → Admin le gère

Recycle

Disque nettoyé, réutilisable (obsolète)

Production: TOUJOURS utiliser Retain, puis supprimer manuellement après backup.

StatefulSet vs Deployment

Identité	Interchangeable (random names)	Stable (mysql-0, mysql-1, mysql-2)
Ordre de démarrage	Parallèle (tous lancés à la fois)	Séquentiel (0 → 1 → 2)
Déploiement rolling	Tous les pods simultanément	Un par un (backward)
Stockage	Pas de PVC propre à chaque pod	volumeClaimTemplates (PVC per pod)
Service	ClusterIP (LoadBalancer)	Headless (DNS direct)
Cas d'usage	Stateless apps (web, API)	Stateful (DB, cache cluster)

⚠ StatefulSet = complexité : ordonnancement, affinity, scale down risqué (orphan data)

StatefulSet avec volumeClaimTemplates

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql # Headless Service
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels: { app: mysql }
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        volumeMounts:
        - name: mysql-data
          mountPath: /var/lib/mysql
  volumeClaimTemplates: # ← AUTO-CREATE PVC
  - metadata:
      name: mysql-data
    spec:
      accessModes: [ ReadWriteOnce ]
      storageClassName: standard-rwo
      resources:
        requests:
          storage: 10Gi
```

serviceName

Headless Service name

volumeClaimTemplates

K8s crée mysql-0-pvc, mysql-1-pvc...

volumeMounts

Mont dans le container

Résultat :

PVC 'mysql-0-mysql-data': 10Gi (mysql-0)
PVC 'mysql-1-mysql-data': 10Gi (mysql-1)
PVC 'mysql-2-mysql-data': 10Gi (mysql-2)

PostgreSQL dans K8s — Architecture

Headless Service

postgres-0.postgres

postgres-0
Primary
R/W

→ ↓ PVC ↓ → pg-0-disk

postgres-1
Replica
R-only

→ ↓ PVC ↓ → pg-1-disk

postgres-2
Replica
R-only

→ ↓ PVC ↓ → pg-2-disk

Architecture

✓ Headless Service

✓ 3 replicas (master+slaves)

✓ DNS discovery

✓ Replication auto

✓ PVC per pod

✓ Init containers pour schema setup

⚠ Manage backup séparément (CronJob + pg_dump)

En production : Cloud SQL vs Self-hosted

Aspect	Self-hosted (StatefulSet)	Cloud SQL (Managed)
HA / Failover	Manuel ou opérateur complexe	Auto (managed 99.95% SLA)
Backup	Votre CronJob + pg_dump	Auto 24h retention
Patching	Vous gerez les versions	Auto (maintenance windows)
Monitoring	Prometheus/Grafana + opérations	GCP console intégré
Encryption	Vous chiffrez les disks	Encryption at rest + TLS
Coût	Cheap disks, expensive ops	Higher, mais 0 ops overhead

RFLKT Reality: Cloud SQL + Cloud SQL Proxy (sidecars) + Private Service Connection

Pièges production — Data loss scenarios

ReclaimPolicy Delete (défaut)

kubectl delete pvc → PV supprimé → DATA PERDUE

⚙ *Fix: Changer ReclaimPolicy: Retain, supprimer manuel après backup*

PVC supprimée par erreur

StatefulSet scale down, puis 'rm pvc' → orphan PV + data

⚙ *Fix: Toujours backup avant ops. Monitoring: PVC existence check*

StatefulSet scale down → orphan PVC

Scale 5 → 3 : PVC 3,4 pas supprimées (par design). Mais : unused storage $$

⚙ *Fix: Monitor et cleanup manuel. Document: scale down = data risk*

No backup strategy

Disque corrompu, GCP outage → 0 recovery RTO/RPO

⚙ *Fix: Backup CronJob (pg_dump → GCS). Restore test mensuel*

Architecture du TP

Namespace: {student-name}

Traefik Ingress

→

Frontend (nginx:1.25)
Deployment

API (Go)
Deployment

PostgreSQL
StatefulSet (1 replica)

→

Headless Service

Workflow du TP

1. Créer Headless Service
2. Créer StatefulSet PostgreSQL (1 replica) avec volumeClaimTemplate
3. Vérifier la PVC créée automatiquement
4. Pod doit accéder à la DB via service DNS (postgres-0.postgres:5432)
5. Init container : créer une table users, insérer des données
6. API + Frontend accèdent à la DB via Deployment + Service ClusterIP
7. Traefik expose Frontend + API

Démo live : RFLKT Cloud SQL Pattern

Cloud SQL Instance

Managed PostgreSQL 17

Private IP (10.0.1.10)

High Availability (1 primary + 2 standbys)

Auto-backup 7 jours

⚠ Not in Kubernetes

Pod → Proxy

Cloud SQL Proxy (sidecar)

Pod container A: main app

Pod container B: cloud-sql-proxy

Proxy: localhost:5432 → Cloud SQL

Authentication: Workload Identity

Connection pool: PgBouncer

→ Cloud SQL

Terraform Module

modules/database/main.tf

GCP Cloud SQL resource

Private Service Connection

Terraform secrets (passwords)

Outputs: internal IP, port

```
modules/database/main.tf→ resource "google_sql_database_instance"
```

Recap & Mini-défi

4 Takeaways clés

1. K8s pods = éphémères. Disques = persistants. PVC = le lien.

2. StatefulSet + volumeClaimTemplate = PostgreSQL HA en K8s.

3. En production : Cloud SQL managed > StatefulSet self-hosted (moins d'ops).

4. ReclaimPolicy: Retain obligatoire. Backup strategy distinct de persistance.

Mini-défi bonus

1. Ajouter un init container au StatefulSet PostgreSQL.

2. Ce container : check si PostgreSQL est up (pg_isready).

3. Une fois up : exécuter un script SQL (CREATE TABLE, INSERT données).

4. Le pod principal attend que l'init finisse avant de démarrer.

5. Bonus: Écrire le script SQL dans une ConfigMap.

Session 5 — ConfigMaps et Secrets (externaliser la config + data sensible)